Comprehensive Documentation: Arabic Video Transcript Analysis Using Large Language Models

[Your Name]

May 25, 2025

Abstract

This project develops a sophisticated Python-based system to process video transcripts using Large Language Models (LLMs), producing Arabic summaries, keywords, translated transcripts, test questions, and interactive chatbot responses. Integrated into a pipeline of Video → Speech-to-Text (SST) → LLM → Text-to-Speech (TTS) → Video, the system leverages Groqs cloud-based LLMs and Ollamas local models for flexible, scalable processing. The project was executed in nine meticulously planned phases, from foundational research to code integration, utilizing Object-Oriented Programming (OOP) and inheritance for modularity and extensibility. Five models were tested, with Qwen-qwq-32B and Llama 4 variants (Scout and Maverick) demonstrating superior performance. This document details the projects significance, methodology, model comparisons, data flow, techniques, development phases, and future enhancements, positioning it as a transformative tool for Arabic multimedia processing and education.

1 Introduction

The proliferation of video content in education, entertainment, and professional domains has created a demand for automated tools to analyze and enhance accessibility, particularly for underrepresented languages like Arabic. This graduation project focuses on the Large Language Model (LLM) phase of a video processing pipeline, transforming raw video transcripts into structured Arabic outputs such as summaries, keywords, translated transcripts, test questions, and chatbot responses. By leveraging state-of-the-art LLMs through Groqs cloud API and Ollamas local framework, the system ensures robust performance and adaptability. The project was developed through nine distinct phases, employing Object-Oriented Programming (OOP) and inheritance to create a modular, reusable codebase. This documentation provides a comprehensive overview of the project, highlighting its significance, technical implementation, model evaluations, and future potential.

2 Significance of the LLM Phase

The LLM phase is the analytical cornerstone of the video processing pipeline, enabling the transformation of raw transcript data into meaningful, user-friendly outputs. Its importance lies in:

- **Accessibility**: Generates Arabic summaries and translated transcripts, making video content accessible to Arabic-speaking audiences.
- **Educational Value**: Produces test questions and answers, facilitating learning and assessment in academic settings.
- **Interactivity**: Offers a chatbot interface for dynamic user queries, enhancing engagement with video content.
- **Efficiency**: Automates complex natural language processing tasks, reducing manual effort in content analysis.
- **Scalability**: Supports both cloud-based (Groq) and local (Ollama) LLMs, ensuring flexibility for different computational environments.

This phase bridges the gap between raw transcript data and actionable outputs, making it indispensable for the pipelines success and its application in education, media, and accessibility.

3 Project Development Phases

The project was executed in nine carefully structured phases, each building on the previous to create a robust system:

1. **Phase 0: Research on LLMs and Transformers**
    - **Objective**: Understand LLMs, Transformers, and their applications.
    - **Details**: Studied the architecture of Transformers, the foundation of modern LLMs, which use self-attention mechanisms to process sequential data. Compared BERT (bidirectional, suited for tasks like classification) and GPT-based models (autoregressive, ideal for generation tasks like summarization). Learned about LLM deployment options, including cloud APIs (Groq) and local frameworks (Ollama).
    - **Outcome**: Gained insights into model selection and API integration, guiding subsequent development.
2. **Phase 1: Initial Code Without Functions**
    - **Objective**: Create a basic script to test LLM capabilities.
    - **Details**: Wrote a simple Python script to process a sample transcript using Ollamas Llama 3.1 7B model, focusing on basic text input and output without structured functions.
    - **Outcome**: Established a proof-of-concept but recognized the need for modularity due to code redundancy.
3. **Phase 2: Testing with Ollama and Llama 3.1 7B**
    - **Objective**: Evaluate a local LLM for transcript processing.
    - **Details**: Used Ollama to run Llama 3.1 7B locally, testing its ability to generate Arabic summaries and keywords from sample transcripts. Observed moderate performance but slow processing due to local hardware constraints.

- **Outcome**: Confirmed feasibility of local LLMs but identified the need for cloud-based alternatives for efficiency.

4. **Phase 3: Writing Functions**
   - **Objective**: Modularize the code for reusability.
   - **Details**: Refactored the script into functions for summarization, keyword extraction, and transcript translation, improving code organization and reducing duplication.
   - **Outcome**: Enhanced maintainability, setting the stage for class-based design.
5. **Phase 4: Developing a Class**
   - **Objective**: Encapsulate functionality in a class using OOP principles.
   - **Details**: Created the `Full_LLM` class to handle transcript processing tasks, incorporating methods like `Summarize`, `Keywords`, and `Questions_Answers`. Used OOP to ensure data encapsulation and method organization.
   - **Outcome**: Produced a structured, reusable codebase suitable for expansion.
6. **Phase 5: Integrating Groq for Cloud-Based Model Testing**
   - **Objective**: Test multiple LLMs without local downloads.
   - **Details**: Integrated Groqs API to test five models: Llama 4 Scout 17B 16E, Llama 4 Maverick 17B 128E, Llama 70B, ALLaM-7B, and Qwen-qwq-32B. Evaluated their performance on Arabic summarization, keyword extraction, and question generation using sample transcripts.
   - **Outcome**: Identified Qwen-qwq-32B and Llama 4 variants (Scout and Maverick) as top performers due to their fluency and accuracy in Arabic.
7. **Phase 6: Creating a Dedicated Groq Class**
   - **Objective**: Isolate Groq-specific functionality.
   - **Details**: Developed the `Groq_Env` class to handle Groq API interactions, supporting streaming (`Groq_chat_answer_stream`) and non-streaming (`Groq_chat_answer`) responses. This class encapsulated API key management and model configuration.
   - **Outcome**: Improved modularity by separating cloud-based LLM logic from general processing.
8. **Phase 7: Implementing Inheritance**
   - **Objective**: Enhance code structure using OOP inheritance.
   - **Details**: Extended `Groq_Env` with `Full_LLM` to support both Groq and Ollama models. Inheritance allowed `Full_LLM` to reuse `Groq_Env`s methods while adding task-specific functionality (e.g., `Summarize`, `Transcript`).
   - **Outcome**: Achieved a flexible, scalable design that supports multiple LLM backends.

9. **Phase 8: Testing the Code**
    - **Objective**: Validate system functionality.
    - **Details**: Tested the `Full_LLM` class with sample transcripts, evaluating outputs for accuracy, fluency, and task completion. Compared model performance across tasks like summarization and question generation.
    - **Outcome**: Confirmed that Qwen-qwq-32B and Llama 4 variants excelled, while ALLaM-7B underperformed.
10. **Phase 9: Integration with Pipeline**
    - **Objective**: Connect the LLM phase with SST and TTS components.
    - **Details**: Integrated the `Full_LLM` class with the Speech-to-Text (SST) output (English transcripts with timestamps) and Text-to-Speech (TTS) input (Arabic summaries, translations, etc.). Ensured seamless data flow by standardizing input/output formats.
    - **Outcome**: Created a cohesive pipeline: Video $\rightarrow$ SST $\rightarrow$ LLM $\rightarrow$ TTS $\rightarrow$ Video.

4 Use of Object-Oriented Programming and Inheritance

Object-Oriented Programming (OOP) was a cornerstone of the projects design, providing:

- **Encapsulation**: Grouped related data (e.g., API keys, model names) and methods (e.g., `Summarize`, `Keywords`) within classes, reducing complexity and improving security.
- **Modularity**: Enabled independent development and testing of components (e.g., `Groq_Env` for API calls, `Full_LLM` for tasks).
- **Reusability**: Allowed reuse of methods across different tasks and models.

Inheritance was used to extend `Groq_Env` with `Full_LLM`, enabling:

- **Code Reuse**: `Full_LLM` inherited Groqs API functionality, reducing duplication.
- **Extensibility**: Added support for Ollama models without modifying `Groq_Env`.
- **Flexibility**: Allowed seamless switching between cloud and local models via the `online` flag.

This OOP-based design ensured a scalable, maintainable system, critical for integrating new models or tasks in the future.

5 Data Flow in the Pipeline

The project operates within a five-stage pipeline:

1. **Video Input**: A video file is processed to extract audio.
2. **Speech-to-Text (SST)**: The audio is converted into an English transcript with timestamps (e.g., `00:00:01 Hello, this is a video about AI`).

3. **LLM Processing**: The transcript is fed into the `Full_LLM` class, which:
    - Cleans timestamps (if needed) using `del_time`.
    - Processes the transcript using predefined Arabic prompts for tasks like summarization, keyword extraction, translation, question generation, or chatbot responses.
    - Interfaces with Groq (Qwen-qwq-32B, Llama 4 variants) or Ollama (Llama 3.1 7B) models.
4. **Text-to-Speech (TTS)**: Arabic outputs (e.g., summaries, translated transcripts) are converted to audio.
5. **Video Output**: The audio is synchronized with the original video to produce a dubbed or annotated version.

6 Techniques Employed

The project utilized a range of advanced techniques:

- **API Integration**: Leveraged Groqs cloud API for high-performance LLM inference and Ollama for local model execution.
- **Prompt Engineering**: Designed precise Arabic prompts to ensure culturally and linguistically appropriate outputs, e.g., `"Generate a summary in Arabic"`.
- **Text Preprocessing**: Implemented `del_time` to remove timestamps, later enhanced with regex (`r'\d{2}:\d{2}:\d{2}'`) for accuracy.
- **Streaming Responses**: Supported real-time output generation for interactive applications.
- **Object-Oriented Programming**: Used classes and inheritance for modularity and scalability.
- **Multi-Model Support**: Enabled seamless switching between Groq and Ollama models.
- **Natural Language Processing**: Applied LLMs for summarization, keyword extraction, translation, and question generation.

7 Model Comparison

Five models were tested: Llama 4 Scout 17B 16E, Llama 4 Maverick 17B 128E, Llama 70B, ALLaM-7B, and Qwen-qwq-32B. Evaluation criteria included Arabic fluency, response accuracy, coherence, and processing speed.

Findings:

- **Qwen-qwq-32B**: Delivered the best performance, producing fluent, accurate Arabic outputs across all tasks. Its larger parameter size enabled superior handling of complex tasks like question generation.

Table 1: Model Comparison for Arabic Transcript Processing

Model	Quality (Arabic Fluency)	Speed	Comme
Llama 4 Scout 17B 16E	Excellent	Fast	Highly fluent, accurate, ide
Llama 4 Maverick 17B 128E	Excellent	Very Fast	Exceptional coherence, be
Llama 70B	Good	Moderate	Reliable but slower, mi
ALLaM-7B	Poor	Slow	Inconsistent, poor Arabic
Qwen-qwq-32B	Outstanding	Very Fast	Best overall, robust

- **Llama 4 Scout 17B 16E and Maverick 17B 128E**: Both excelled in fluency and speed, with Maverick slightly faster due to its optimized architecture. Ideal for real-time applications.
- **Llama 70B**: Performed well but was slower and less fluent than Llama 4 variants or Qwen.
- **ALLaM-7B**: Produced inconsistent, poorly structured Arabic outputs, making it unsuitable for the pipeline.

8 Future Work

To enhance the system, the following advancements are planned:

- **Retrieval-Augmented Generation (RAG)**: Integrate RAG to combine transcript data with external knowledge bases, improving response accuracy and contextuality.
- **Historical Conversation**: Implement memory to retain conversation context, enabling coherent, multi-turn chatbot interactions.
- **Prompt Class**: Develop a dedicated class for dynamic prompt engineering, supporting customizable prompts for multiple languages and tasks.
- **Multilingual Support**: Extend the system to process transcripts in additional languages (e.g., French, Spanish).
- **Web Interface**: Build a user-friendly interface using Flask or Django to allow transcript uploads and interactive output visualization.
- **Automated Evaluation**: Implement metrics like BLEU for translation quality and ROUGE for summarization to quantify performance.

9 Recommendations for Enhancement

To elevate the project for academic and practical impact:

- **Robust Error Handling**: Add try-catch blocks to handle API failures, invalid inputs, or network issues.
- **Security Enhancements**: Use environment variables (`os.environ.get("GROQ_API_KEY")`) to secure API keys.

- **Performance Metrics**: Quantify model performance using NLP metrics (e.g., BLEU, ROUGE) and response times.
- **Automated Testing**: Develop a test suite to validate outputs across diverse transcripts and models.
- **Visualization**: Create keyword clouds or summary visualizations to enhance presentation impact.
- **Documentation Expansion**: Include code snippets and sample outputs in an appendix to demonstrate functionality.

10 Conclusion

This project delivers a robust, modular system for processing video transcripts using LLMs, producing high-quality Arabic outputs for accessibility and education. Executed through nine phases, from research to pipeline integration, the system leverages OOP and inheritance for scalability and maintainability. Testing revealed Qwen-qwq-32B and Llama 4 variants as top performers, while ALLaM-7B was inadequate. The projects integration into the Video $\rightarrow$ SST $\rightarrow$ LLM $\rightarrow$ TTS $\rightarrow$ Video pipeline demonstrates its practical utility. Future enhancements like RAG, historical conversation, and a Prompt Class will further elevate its capabilities, positioning it as a transformative tool for multimedia processing in Arabic-speaking communities.